Introduction to Deep Equilibrium Nets (DEQNs)

A Mini-Course

Galo Nuño Simon Scheidegger

Banco de España

HEC, University of Lausanne

March 2026

Based on: Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.
DOI: https://doi.org/10.1111/iere.12575

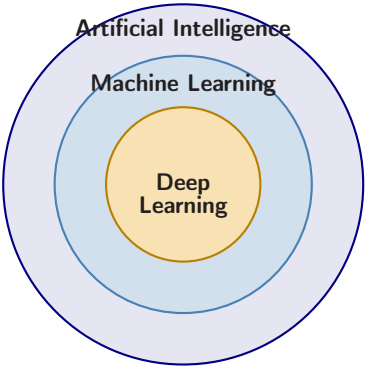
Materials: https://github.com/sischei/DEQN_Galo_03_2026

Course Outline

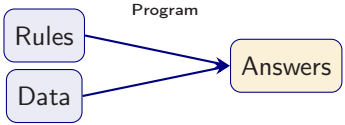
1. ML/DL Foundations
2. Deep Equilibrium Nets
3. Brock–Mirman Benchmark
4. Practical Tools: NAS & Loss Normalization
5. Scaling Up — The IRBC Model
6. Hands-on Exercises

Outline

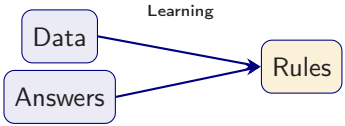
What is Machine Learning?



Classical programming:

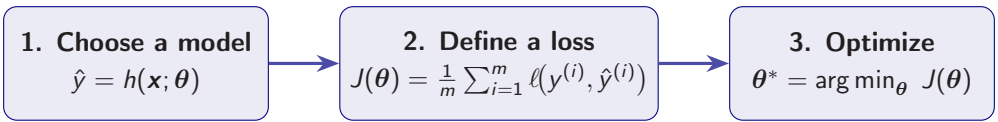


Machine learning:



The Machine Learning Recipe

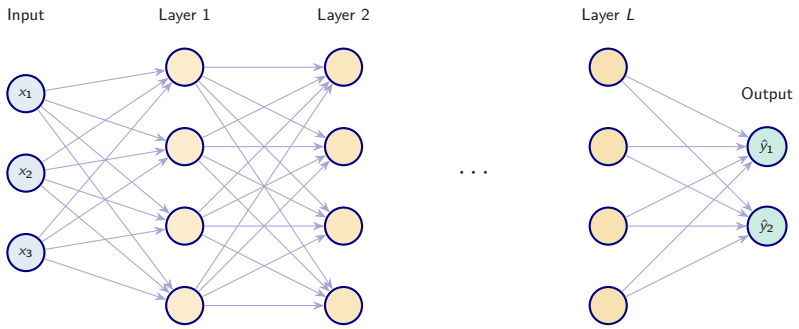
Every supervised learning algorithm follows three steps:



Cost function example (Mean Squared Error)

$$J(\boldsymbol{\theta}) \;=\; \frac{1}{2m} \sum_{i=1}^{m} \big(h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)}\big)^2$$

Deep Feedforward Networks



An L-layer network computes a nested composition:

$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_L\Big(\boldsymbol{W}^{(L)} g_{L-1}\big(\cdots g_1\big(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big)\cdots + \boldsymbol{b}^{(L-1)}\big) + \boldsymbol{b}^{(L)}\Big)$$

Each layer transforms the representation. **Depth** provides exponential gains in representational power over width alone.

Universal Approximation Theorem

Theorem (Cybenko 1989; Hornik, Stinchcombe & White 1989)

Let $g : \mathbb{R} \to \mathbb{R}$ be a bounded, non-constant, continuous activation function. Then for any continuous function $f : \mathcal{K} \to \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a single-hidden-layer network

$$F(\boldsymbol{x}) = \sum_{j=1}^{n_1} v_j \, g(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$$

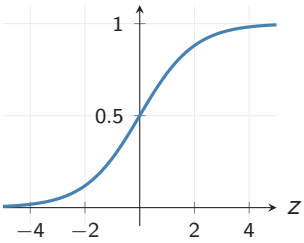
such that $\sup_{\boldsymbol{x} \in \mathcal{K}} |F(\boldsymbol{x}) - f(\boldsymbol{x})| < \varepsilon$.

Caveats:

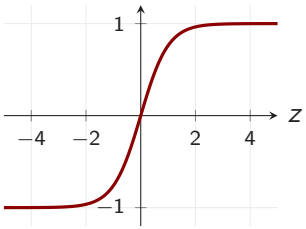
- The required width n_1 may be **exponentially large** in d.
- Existence $\neq$ efficient learnability.
- Practical success of deep learning suggests that **depth** provides a much more efficient parameterization than width alone.

Activation Functions

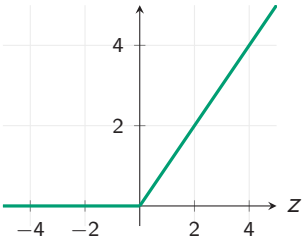
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$



Tanh: $\tanh(z)$

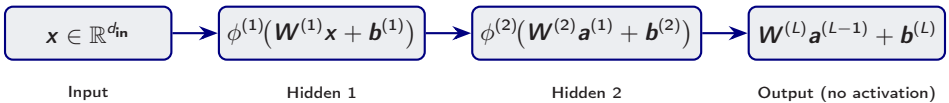


ReLU: $\max(0, z)$



Other common activations: Leaky ReLU $\max(0.01z, z)$, Softplus $\ln(1 + e^z)$, ELU, Swish $z \cdot \sigma(z)$.

DNN Layer Structure



Each hidden layer applies a nonlinear activation $\phi^{(\ell)}$:

$$a^{(\ell)} = \phi^{(\ell)}\big(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}\big), \qquad \ell = 1, \ldots, L-1$$

- The output layer is typically linear (for regression) or uses softplus (to enforce positivity, e.g. consumption)
- The full network: $\mathcal{N}_\rho(x) = W^{(L)}a^{(L-1)} + b^{(L)}$

Gradient Descent and SGD

Goal: find weights that minimize the loss:

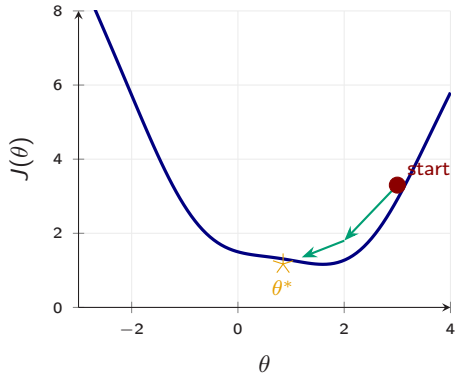
$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Strategy: repeatedly step in the direction of steepest descent:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Mini-batch SGD: sample a batch $\mathcal{B}$ of B examples:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{B} \sum_{k \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} J_k(\boldsymbol{\theta})$$



Loss Functions

The loss function $J(\boldsymbol{\theta})$ measures how far predictions $\hat{\boldsymbol{y}}$ are from targets $\boldsymbol{y}$.

Regression: Mean Squared Error

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(y^{(i)} - f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}) \right)^2$$

Penalizes large deviations quadratically.

Classification: Cross-Entropy

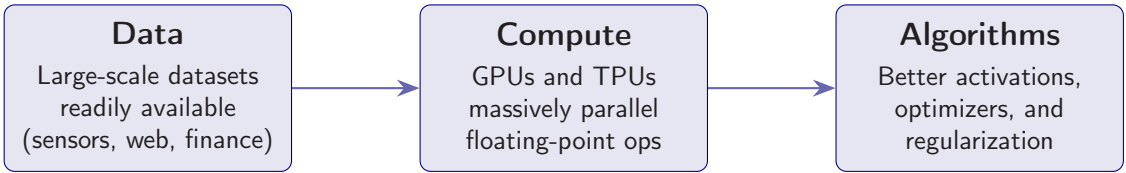
$$J = -\frac{1}{n} \sum_{i=1}^{n} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

For binary labels $y \in \{0, 1\}$.

In computational economics, we will replace labeled data with structural equations—the key insight behind DEQNs.

Why Deep Learning Now?

Neural networks date back to the 1940s. Three developments made them practical:



Key milestones: Perceptron (1958), Backpropagation (1986), Deep CNN (1995), AlexNet (2012), Transformers (2017), ChatGPT (2022).

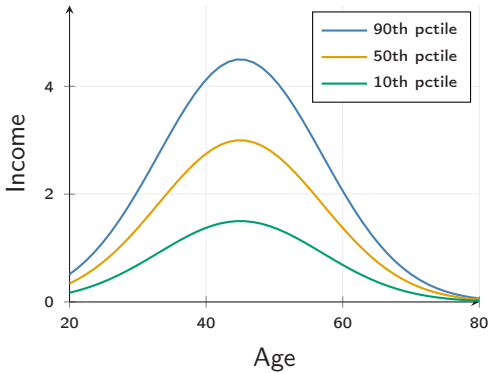
Outline

Heterogeneity Matters

Key questions in macroeconomics:

- How does inequality evolve over the life cycle?
- How do idiosyncratic shocks interact with aggregate risk?
- What are the distributional effects of policy?

$\Rightarrow$ Need overlapping-generations (OLG) models with many heterogeneous agents.



Stylized income-by-age profiles

Dynamic Stochastic General Equilibrium Models

Generic formulation: find policy functions $p(\cdot)$ such that

$$\mathbb{E}\Big[G\big(\boldsymbol{x}_t,\, \boldsymbol{x}_{t+1},\, p(\boldsymbol{x}_t),\, p(\boldsymbol{x}_{t+1})\big) \;\Big|\; \boldsymbol{x}_t,\, p(\boldsymbol{x}_t) \Big] = 0$$

with transition law $\boldsymbol{x}_{t+1} = h\big(\boldsymbol{x}_t,\, p(\boldsymbol{x}_t),\, \varepsilon_{t+1}\big)$.

Challenges:

- State space $\boldsymbol{x}_t \in \mathbb{R}^d$ can be very high-dimensional
- Standard grid-based methods scale as $\mathcal{O}(n^d)$ $\Rightarrow$ curse of dimensionality
- Need a method that does not rely on grids

What is High-Dimensional?

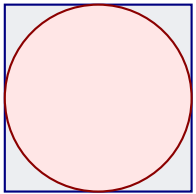
Dimensions d	Grid points n^d	Runtime
1	10	10 seconds
2	100	2 minutes
5	10^5	1 day
10	10^{10}	317 years
20	10^{20}	$\sim$ 3 trillion years

Assumes $n = 10$ grid points per dimension and 1 second per evaluation.

Three remedies:

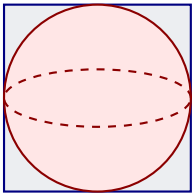
1. Sparse grids (Brumm & Scheidegger, 2017)
2. Random grids (Rust, 1997)
3. Neural networks (this lecture)

Volumes in High Dimensions



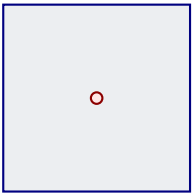
2D

$V_{\text{sphere}}/V_{\text{cube}} = \pi/4 \approx 0.785$



3D

$V_{\text{sphere}}/V_{\text{cube}} = \pi/6 \approx 0.524$



100D

$V_{\text{sphere}}/V_{\text{cube}} \approx 1.87 \times 10^{-70}$

Implication

In high dimensions, almost all volume is in the corners. Grid-based methods waste most of their evaluation points in regions that barely contribute to the integral or approximation.

Our Solution: Deep Equilibrium Nets

Core idea (Azinovic, Gaegauf & Scheidegger, 2022):

Replace the traditional grid-based solution with a neural network trained to satisfy economic equilibrium conditions.

Three Key Ingredients

1. **Equilibrium error as loss function:** use the economic equations $G(\cdot) = 0$ directly—no labeled data needed
2. **Stochastic gradient descent:** optimize network parameters ρ via SGD on mini-batches
3. **Simulated paths:** draw training points from the ergodic distribution of the model itself

$\Rightarrow$ This turns solving an economic model into training a neural network.

What is a Deep Neural Network?

A deep neural network (DNN) $\mathcal{N}_\rho : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is a parametric function approximator.

Universal approximation (Cybenko 1989, Hornik 1991):

$$\text{For any continuous } f : \mathcal{K} \to \mathbb{R} \text{ and } \varepsilon > 0, \quad \exists\, \rho^\star : \quad \|\mathcal{N}_{\rho^\star} - f\| < \varepsilon$$

In economics:

- Input: state variables $\boldsymbol{x}_t \in \mathbb{R}^d$ (capital, productivity, wealth distribution, ...)
- Output: policy variables (consumption, savings, prices, ...)
- Parameters ρ: weights and biases—found by training

The "Data Problem" in Economics

Deep learning excels when massive labeled datasets are available.

But in computational economics:

- We solve functional equations, not classification/regression tasks
- There are no labeled pairs $(\boldsymbol{x}_i, y_i)$
- Traditional approach: time iteration on a grid—but grids do not scale

Key insight:

- We know the structural equations $G(\cdot) = 0$
- We can check how well a candidate policy satisfies them
- $\Rightarrow$ Use the equilibrium error as the loss function!

Traditional Approach: Time Iteration on a Grid

Algorithm: Grid-Based Collocation

Input: Grid $\mathcal{G} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M\} \subset \mathbb{R}^d$, initial guess $p^{(0)}$
for $k = 0, 1, 2, \ldots$ until convergence **do**
  for each grid point $\boldsymbol{x}_i \in \mathcal{G}$ **do**
    Solve $G\big(\boldsymbol{x}_i,\, p^{(k)}(\boldsymbol{x}_i),\, p^{(k)}\big) = 0$ for $p^{(k+1)}(\boldsymbol{x}_i)$
  end for
  Interpolate $p^{(k+1)}$ between grid points
end for

Problems:

- Grid size $M = n^d \Rightarrow$ exponential in dimension d
- Interpolation in $d > 5$ becomes expensive and inaccurate
- Cannot handle models with $d = 60$ or more state variables

The DEQN Loss Function

Key idea: replace labeled data with economic equilibrium conditions.

Let $G(\boldsymbol{x}, f(\boldsymbol{x})) = 0$ be the system of equilibrium equations. Define:

$$\ell_\rho = \frac{1}{N} \sum_{i=1}^{N} \left\| G\left(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i)\right) \right\|^2$$

- $\mathcal{N}_\rho(\boldsymbol{x}_i)$ is the neural net's predicted policy at state $\boldsymbol{x}_i$
- $G(\cdot)$ measures the equilibrium error (e.g. Euler equation residual)
- $\ell_\rho = 0$ iff the policy exactly satisfies all equilibrium conditions

$\Rightarrow$ No labels needed—this is **unsupervised** (or "self-supervised") learning.

Training Algorithm for DEQNs

Algorithm 1: Deep Equilibrium Net Training

Input: Initial state $\boldsymbol{x}_0$, network $\mathcal{N}_\rho$, learning rate α, # episodes E, # epochs T
for episode $e = 1, \ldots, E$ **do**
  Simulate path: $\boldsymbol{x}_0 \to \boldsymbol{x}_1 \to \cdots \to \boldsymbol{x}_T$ using $\mathcal{N}_\rho$ and transition law
  for epoch $t = 1, \ldots, T$ **do**
    Draw mini-batch $\mathcal{B} \subset \{\boldsymbol{x}_0, \ldots, \boldsymbol{x}_T\}$
    Compute loss: $\ell_\rho = \frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x}_i \in \mathcal{B}} \| G(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i)) \|^2$
    Update: $\rho \leftarrow \rho - \alpha \cdot \nabla_\rho \ell_\rho$
  end for
end for
Output: Trained network $\mathcal{N}_{\rho^\star}$ approximating the policy function

Note: Training points come from the model's own simulation—the network learns on the ergodic distribution, focusing on economically relevant regions.

Supervised vs. Unsupervised (DEQN) Learning

Key difference: DEQNs replace labeled data with structural economic equations.

The Cost of Labeled Data in Economics

Supervised learning requires labeled pairs $(\boldsymbol{x}_i, y_i)$.

In computational economics, generating a single label y_i means solving the model at state $\boldsymbol{x}_i$—precisely the problem we are trying to solve!

Step	Supervised (label generation)	DEQN (unsupervised)
1	Choose grid $\mathcal{G}$ of n^d points	Draw states from simulation
2	Solve $G(\boldsymbol{x}_i, y_i) = 0$ at each $\boldsymbol{x}_i$	Evaluate residual $G(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i))$
3	Fit $\hat{y} = f(\boldsymbol{x}; \boldsymbol{\theta})$ to $\{(\boldsymbol{x}_i, y_i)\}$	Update ρ via SGD on residual
Cost	$\underbrace{n^d}_{\text{grid}} \times \underbrace{C_{\text{solve}}}_{\text{per point}} + \text{fitting}$	$E \times T \times \underbrace{B}_{\text{batch}} \times C_{\text{residual}}$

- C_{solve}: cost of solving a nonlinear system or iterating a Bellman equation to convergence
- C_{residual}: cost of a single forward pass through the equilibrium equations—orders of magnitude cheaper

Why Supervised Learning is Prohibitively Expensive

To generate labels, we must solve the model on a grid. This involves:

1. Bellman / value function iteration

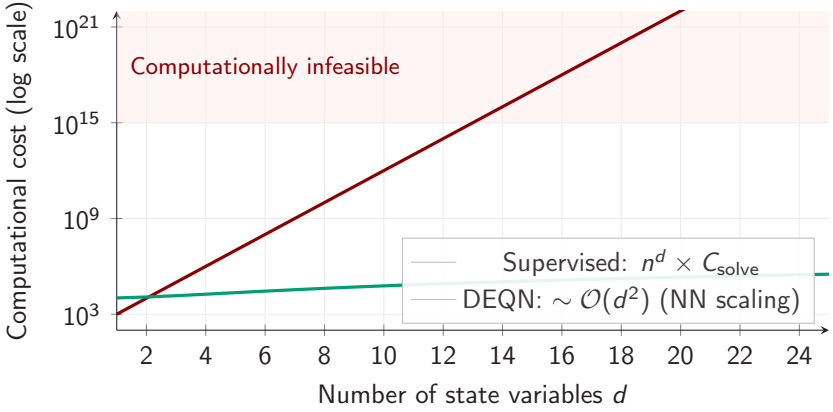
- Iterate $V_{k+1}(\boldsymbol{x}) = \max_a\{u(a) + \beta\mathbb{E}[V_k(\boldsymbol{x}')]\}$
- Each iteration: solve n^d optimisation problems
- Typically $\mathcal{O}(100)$ iterations to converge
- Interpolation between grid points introduces error

2. Nonlinear system solving

- At each $\boldsymbol{x}_i$: solve $G(\boldsymbol{x}_i, y) = 0$ via Newton's method
- Each Newton step: compute and invert a Jacobian
- Cost per point: $\mathcal{O}(m^3)$ for m equations
- Convergence not guaranteed away from the solution

The fundamental bottleneck

Computational Cost: Supervised vs. DEQN



Supervised approaches hit the wall around $d \approx 5$–8. DEQNs scale gracefully to $d > 100$.

DEQN: Key Recap

Replace Labels

Instead of labeled data $(\boldsymbol{x}_i, y_i)$, use the economic equilibrium equations $G(\cdot) = 0$ as the loss.

Replace Grids

Instead of n^d grid points, use a neural network $\mathcal{N}_\rho$ that maps states to policies globally.

Replace TI

Instead of time iteration with interpolation, use SGD on simulated paths from the model.

Result: A method that scales to hundreds of state variables and runs on GPUs.

Outline

Brock–Mirman (1972): Setup

Social planner's problem:

$$\max_{\{C_t\}_{t=0}^{\infty}} \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t \ln(C_t)\right]$$

subject to:

$$K_{t+1} + C_t = z_t K_t^{\alpha}, \qquad \ln z_{t+1} = \varrho \ln z_t + \sigma_z \epsilon_{t+1}, \quad \epsilon_t \sim \mathcal{N}(0,1)$$

Euler equation:

$$\frac{1}{C_t} = \beta\, \mathbb{E}\left[\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}}\right]$$

Analytical solution (log utility, full depreciation $\delta = 1$):

$$K_{t+1} = \beta\alpha\, K_t^{\alpha}, \qquad C_t = (1 - \beta\alpha)\, K_t^{\alpha}$$

DEQN for Brock–Mirman

Step 1: Parametrize the policy function with a neural network:

$$C_t = \mathcal{N}_\rho(K_t, z_t) \qquad \text{(consumption as a function of state)}$$

Step 2: Define the equilibrium residual:

$$G(K_t, z_t) = 1 - \beta \frac{C_t}{C_{t+1}} \cdot \alpha z_{t+1} K_{t+1}^{\alpha-1}$$

where $K_{t+1} = z_t K_t^\alpha - \mathcal{N}_\rho(K_t, z_t)$ and $C_{t+1} = \mathcal{N}_\rho(K_{t+1}, z_{t+1})$.

Step 3: Minimize the DEQN loss:

$$\ell_\rho = \frac{1}{N} \sum_{i=1}^{N} \left| G(K_t^{(i)}, z_t^{(i)}) \right|^2$$

We can verify the solution against the analytical benchmark.

Hands-on Notebooks

Two notebooks accompany this lecture:

1. `01_Brock_Mirman_1972_DEQN.ipynb`
   Deterministic Brock–Mirman with analytical benchmark.
2. `02_Brock_Mirman_Uncertainty_DEQN.ipynb`
   Stochastic version with TFP shocks ($\varrho > 0$, $\sigma_z > 0$).

All notebooks are in the `code/` directory and on the course GitHub repository.

Note: Exercises (`03_DEQN_Exercises_Blancs.ipynb`) build on these notebooks.

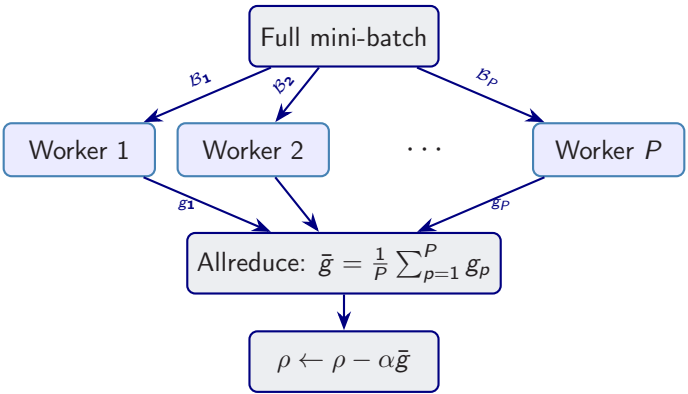
"To pull a bigger wagon, it is easier to add more oxen than to grow a gigantic ox."

— Gropp, Lusk & Skjellum (1999)

Using MPI: Portable Parallel Programming

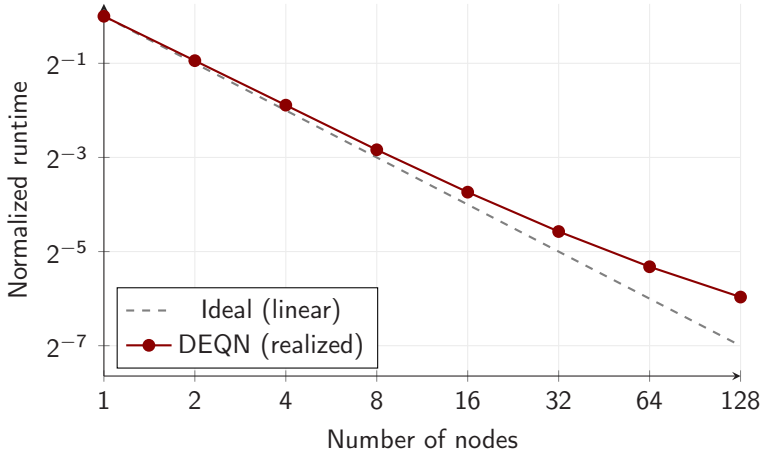
Data Parallelism for DEQNs

Idea: distribute training data across multiple workers (GPUs/CPUs).



Implemented via **Horovod**/MPI. Each worker computes a local gradient; allreduce averages them.

Scalability Results



Near-linear scaling up to 128 nodes. Communication overhead is negligible for typical network sizes.

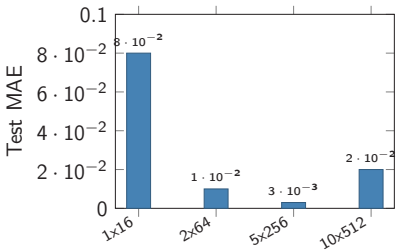
Outline

Why Architecture Matters

Same data, different architectures:

- 1 layer, 16 units $\Rightarrow$ MAE $= 0.08$
- 2 layers, 64 units $\Rightarrow$ MAE $= 0.01$
- 5 layers, 256 units $\Rightarrow$ MAE $= 0.003$
- 10 layers, 512 units $\Rightarrow$ MAE $= 0.02$ (overfit)

Architecture choice can matter more than training duration.



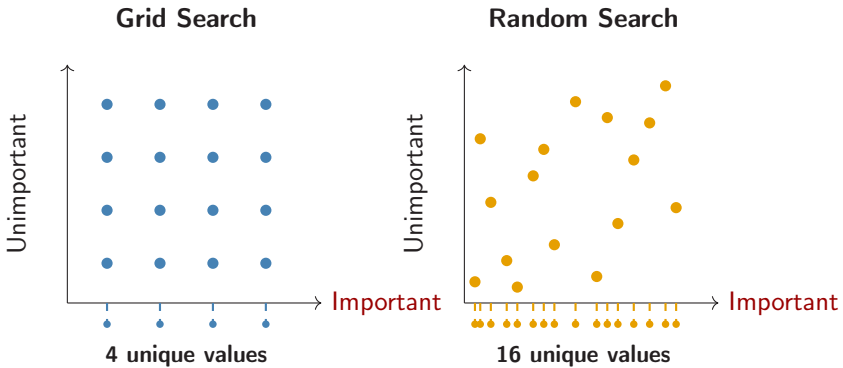
Neural Architecture Search: Method Comparison

Property	Grid	Random	Bayesian	Hyperband
Computational cost	$\mathcal{O}(k^d)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n \log n)$
Sample efficiency	Low	Medium	High	High
Parallelisable	Fully	Fully	Limited	Partially
Handles interactions	No	Partially	Yes	No
Early stopping	No	No	No	Yes
Implementation	Trivial	Trivial	Moderate	Moderate

Practical advice:

- **Quick exploration:** Random search (strong baseline)
- **Limited budget:** Bayesian optimisation (sample-efficient)
- **Large search space:** Hyperband (aggressive early stopping)

Grid vs. Random — Why Random Wins



When only one dimension matters, grid wastes $\frac{3}{4}$ of its budget on the *unimportant* axis. Random explores the important axis $4\times$ more densely.

Bayesian Optimisation

Idea: Build a *surrogate model* of $f(\text{HP}) \rightarrow$ loss and use it to choose the next trial *intelligently*.

1. Fit a Gaussian process (GP) to observed trials
2. Compute an acquisition function (e.g., Expected Improvement)
3. Evaluate the HP with highest acquisition value
4. Update the GP and repeat

Key advantage: Sample-efficient — exploits structure in the response surface.

Expected Improvement

$$\text{EI}(\boldsymbol{x}) = \mathbb{E}[\max(f^* - f(\boldsymbol{x}), 0)]$$

- f^*: best observed value
- Balances *exploration* (high uncertainty) and *exploitation* (low predicted loss)

NAS for Computational Economics

Where NAS matters in computational economics:

1. **Surrogate models:** Replace expensive simulations (e.g., climate IAMs) with NNs — architecture determines approximation quality.
2. **DSGE policy functions:** Deep equilibrium nets use NNs to approximate policy/value functions. Architecture search over the policy network can improve convergence.
3. **High-dimensional problems:** As d grows (many state variables), the architecture must scale. NAS automates this adaptation.
4. **Reproducibility:** Automated search documents *which* architectures were tried, making results more transparent than "we chose 3 layers with 64 units."

Multi-Component Losses in DEQNs

Many problems require minimising *several* loss terms simultaneously:

Deep equilibrium nets (DEQNs):

- Euler equation residuals
- Budget constraints
- Market clearing

Physics-informed NNs (PINNs):

- PDE residual $\mathcal{L}_{\text{PDE}}$
- Boundary conditions $\mathcal{L}_{\text{BC}}$
- Initial conditions $\mathcal{L}_{\text{IC}}$

Core challenge

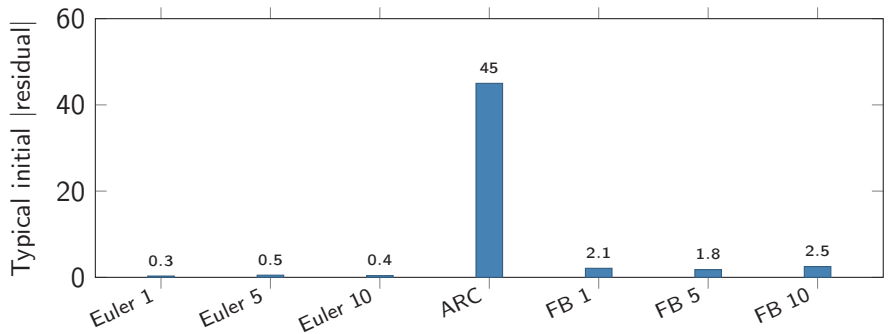
These loss components often live at vastly different scales.

With equal weights:

- Large-scale component dominates gradients
- Small-scale component is ignored
- The network effectively solves a *single-task* problem

The Scale Mismatch Problem

Example: IRBC model with $N = 10$ countries has three types of residuals:



- The aggregate resource constraint (ARC) is $\sim 100\times$ larger than the Euler residuals
- With equal weighting: $\nabla_\rho \ell \approx \nabla_\rho \ell_{\text{ARC}}$—the network only learns to clear markets
- Euler equations and complementarity conditions are effectively ignored

Gradient Dominance: Why Naïve Weighting Fails

Total loss: $\ell = \sum_{k=1}^{K} \ell_k$. The gradient is:

$$\nabla_\rho \ell = \sum_{k=1}^{K} \nabla_\rho \ell_k$$

When $\ell_j \gg \ell_k$ for some j:

- $\|\nabla_\rho \ell_j\| \gg \|\nabla_\rho \ell_k\| \;\Rightarrow$ the update direction is dominated by component j
- Components $k \neq j$ may even increase during training
- The network converges to a solution that satisfies *one* equation well but violates the others

Naïve: $w_k = 1$ for all k

- Large-scale loss dominates
- Small losses stagnate or worsen
- Effectively single-task learning

Adaptive: w_k adjusted dynamically

- All loss components contribute equally to the gradient direction
- The network balances all equilibrium

Approaches to Loss Balancing

Method	Idea	Limitation
Equal weights	$w_k = 1/K$	Ignores scale differences entirely
Manual tuning	Hand-pick w_k	Tedious; does not adapt during training
Inverse-loss	$w_k \propto 1/\ell_k$	Oscillates: raises weight on noisy components
GradNorm (Chen et al., 2018)	Match gradient norms across tasks	Requires per-task gradient computation; expensive
Uncertainty weighting (Kendall et al., 2018)	Learn σ_k: $w_k \propto 1/(2\sigma_k^2)$	Extra parameters; can collapse
ReLoBRaLo (Bischof et al.	Relative ratios + random look-back + smoothing	Lightweight; robust; no extra params to learn

ReLoBRaLo — Relative Loss Balancing with Random Lookback

Key innovations (Bischof et al., 2021):

1. **Relative balancing:** Use *ratios* $\mathcal{L}_i^{(t)}/\mathcal{L}_i^{(t-1)}$ instead of absolute values
2. **Random lookback:** With probability ρ, compare to initial losses $\mathcal{L}_i^{(0)}$ instead of the previous step
3. **Exponential smoothing:** Blend new weights with history via α

Why random lookback?

- Prevents weight drift
- "Resets" to global perspective
- Stabilises training
- Like simulated annealing for weights

ReLoBRaLo — Algorithm

Weight update at epoch t

1: Compute losses $\mathcal{L}_1^{(t)}, \ldots, \mathcal{L}_K^{(t)}$

2: **Step-wise weights:** $\hat{w}_i^{(t)} = \text{softmax}_i\left(\frac{\mathcal{L}_i^{(t)}}{T \cdot \mathcal{L}_i^{(t-1)}}\right) \cdot K$

3: **Baseline weights:** $\hat{w}_i^{(0)} = \text{softmax}_i\left(\frac{\mathcal{L}_i^{(t)}}{T \cdot \mathcal{L}_i^{(0)}}\right) \cdot K$

4: **Combine with random lookback:**

$$w_i^{(t)} = \alpha\left[\rho\, w_i^{(t-1)} + (1-\rho)\, \hat{w}_i^{(0)}\right] + (1-\alpha)\, \hat{w}_i^{(t)}$$

- α: smoothing — how much to trust the historical weights
- ρ: lookback probability — how often to reset to initial comparison
- T: temperature — sharpness of the softmax

ReLoBRaLo — Experiment Results

	Equal weights	**Inverse-loss**	**ReLoBRaLo**
f_A mean error	5.2×10^{-1}	8.1×10^{-2}	3.4×10^{-3}
f_B mean error	3.7×10^{0}	1.5×10^{0}	8.2×10^{-1}
f_C mean error	4.5×10^{1}	9.8×10^{1}	5.1×10^{1}
Total rel. error	1.00	0.41	0.12

- **Equal:** f_C dominates training; f_A is essentially not learned
- **Inverse-loss:** Improves f_A, but introduces oscillations
- **ReLoBRaLo:** All three functions well approximated; $\sim 8\times$ lower total error

Loss Balancing for DEQNs

DSGE / Deep equilibrium nets:

- Euler equations at different scales
- Budget constraint residuals
- Market clearing conditions
- Each may have different units and magnitudes

$\Rightarrow$ ReLoBRaLo ensures all equilibrium conditions are learned, not just the largest.

ReLoBRaLo defaults:

Symbol	Name	Default
T	Temperature	1.0
α	Smoothing	0.999
ρ	Lookback prob.	0.999

Practical guidance

The defaults work well in most cases. Only T needs occasional tuning.

Outline

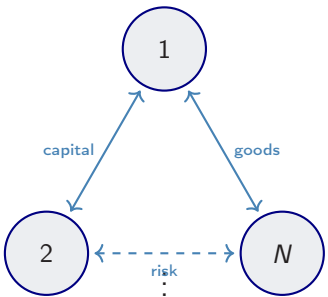
Why International Models?

Key questions in international macro:

- How do capital flows respond to productivity shocks?
- What governs international risk sharing?
- How do trade imbalances emerge?

$\Rightarrow$ Need models with N heterogeneous countries.

$\Rightarrow$ State space: $2N$ dimensions (capital + TFP per country).



Complete markets

IRBC Model Overview

International Real Business Cycle (IRBC) model:

- N countries, each with country-specific capital k^j and TFP z^j
- Complete markets: a social planner allocates resources with Pareto weights τ^j
- Irreversible investment: $I^j \geq 0$ (cannot disinvest)
- Capital adjustment costs: $\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j\prime}}{k^j} - 1 \right)^2$

Dimensionality

State: $\boldsymbol{s} = (k^1, \ldots, k^N, z^1, \ldots, z^N) \in \mathbb{R}^{2N}$
Policy: $(k^{1\prime}, \ldots, k^{N\prime}, \lambda, \mu^1, \ldots, \mu^N) \in \mathbb{R}^{2N+1}$

Preferences and Production

CRRA utility for each country j:

$$u^j(c) = \frac{c^{1-1/\gamma_j}}{1 - 1/\gamma_j}$$

- γ_j: intertemporal elasticity of substitution
- Heterogeneous: $\gamma_j \in [\gamma_{\min}, \gamma_{\max}]$

Social planner maximizes:

$$\max \sum_{t=0}^{\infty} \beta^t \, \mathbb{E}\left[\sum_{j=1}^{N} \tau^j \, u^j(c_t^j)\right]$$

Production in country j:

$$Y^j = A_{\text{tfp}} \cdot \exp(z^j) \cdot (k^j)^{\zeta}$$

TFP process (AR(1)):

$$z^{j\prime} = \rho_z \, z^j + \sigma_e \left(\varepsilon^j + \varepsilon^{\text{agg}}\right)$$

$\beta = 0.99$	$\zeta = 0.36$
$\delta = 0.01$	$\kappa = 0.50$
$\rho_z = 0.95$	$\sigma_e = 0.01$

Equilibrium System

$2N + 1$ **equations in** $2N + 1$ **unknowns:**

- N **Euler equations** (relative error form):

$$\frac{\beta\,\mathbb{E}\left[\lambda' \cdot \mathrm{MPK}^{j\prime} - (1-\delta)\,\mu^{j\prime}\right] + \mu^{j}}{\lambda\,(1+\Gamma_{k'})} - 1 = 0$$

- **Aggregate resource constraint** (1 equation):

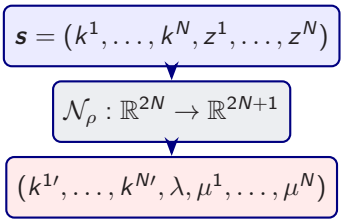
$$\sum_{j=1}^{N}\left[Y^{j} + (1-\delta)\,k^{j} - k^{j\prime} - \Gamma^{j} - c^{j}\right] = 0$$

- N **Fischer–Burmeister conditions** (irreversibility):

$$\mathrm{FB}(\mu^{j}, I^{j}) = \mu^{j} + I^{j} - \sqrt{(\mu^{j})^2 + (I^{j})^2 + \varepsilon} = 0$$

Consumption is determined by λ: $c^{j} = (\lambda/\tau^{j})^{-\gamma_j}$.

DEQN Formulation for the IRBC



Loss: sum of squared residuals over all equilibrium conditions:

$$\ell_\rho = \frac{1}{B} \sum_{i=1}^{B} \left[\sum_{j=1}^{N} \left(\text{Euler}_j^{(i)}\right)^2 + \left(\text{ARC}^{(i)}\right)^2 + \sum_{j=1}^{N} \left(\text{FB}_j^{(i)}\right)^2 \right]$$

Architecture: 2×64 hidden units, Swish activation, Softplus output (ensures positivity).

Scalability in N

N	States	Policies	NN params	Grid pts ($n=10$)
2	4	5	$\sim$4.7K	10^4
4	8	9	$\sim$5.3K	10^8
10	20	21	$\sim$6.7K	10^{20}
50	100	101	$\sim$17K	10^{100}
100	200	201	$\sim$30K	10^{200}

- NN parameters grow linearly in N (only input/output layers scale)
- Grid methods grow exponentially: 10^{2N} points
- DEQN training cost per step: $\mathcal{O}(|\rho| \cdot B \cdot Q^{N+1})$
- For large N: replace tensor-product GH with monomial rules or QMC

From Brock–Mirman to IRBC

	Brock–Mirman	**IRBC**
Countries	1	N
States	(K, z)	$(k^1, \ldots, k^N, z^1, \ldots, z^N)$
Policies	C (or s)	$(k^{1\prime}, \ldots, k^{N\prime}, \lambda, \mu^1, \ldots, \mu^N)$
Equations	1 Euler	N Euler + 1 ARC + N FB
Constraints	none	irreversibility, adj. costs
Shocks	1	$N + 1$
Output act.	sigmoid	softplus
Analytical sol.	yes ($\delta = 1$)	no

The IRBC is a natural scaling test for the DEQN methodology.

IRBC Notebook

Hands-on: `04_IRBC_DEQN.ipynb`

1. Parameters & steady state verification
2. Gauss–Hermite quadrature setup
3. Keras network definition
4. Model equations (production, consumption, adjustment costs)
5. Cost function with Euler, ARC, and FB residuals
6. Two-phase training loop
7. Convergence analysis & policy function plots
8. Economic diagnostics (consumption sharing, trade balance)

Self-contained: all equations implemented inline, no external imports beyond TensorFlow/NumPy.

Outline

Code Notebooks

Notebook	Description
`01_Brock_Mirman_1972_DEQN`	Deterministic Brock–Mirman model with analytical benchmark
`02_Brock_Mirman_Uncertainty_DEQN`	Stochastic Brock–Mirman with TFP shocks
`03_DEQN_Exercises_Blancs`	Exercise problems (blanks to fill in)
`03_DEQN_Exercises_Solutions`	Exercise solutions
`04_IRBC_DEQN`	Full IRBC model: multi-country DEQN with adjustment costs and irreversibility

All notebooks are in the `code/` directory. Requirements: `pip install -r requirements.txt`

Suggested Readings

1. **Azinovic, Gaegauf & Scheidegger** (2022).
   Deep Equilibrium Nets. International Economic Review 63(4), 1471–1525.
2. **Fernández-Villaverde, Nuño & Perla** (2024).
   Taming the Curse of Dimensionality: Quantitative Economics with Deep Learning. NBER Working Paper 33117.
3. **Chen, Didisheim & Scheidegger** (2026).
   Deep Surrogates for Finance: With an Application to Option Pricing. Journal of Financial Economics 177, 104222.
4. **Friedl, Kübler, Scheidegger & Usui** (2023).
   Deep Uncertainty Quantification: With an Application to Integrated Assessment Models. Working paper.

All papers are in the `readings/` directory.

Questions?

Galo Nuño

Banco de España

Simon Scheidegger

HEC, University of Lausanne

https://www.galonuno.com/

https://sites.google.com/site/simonscheidegger/

Materials: https://github.com/sischei/DEQN_Galo_03_2026

Key references:

Azinovic, Gaegauf & Scheidegger (2022)
Deep Equilibrium Nets
International Economic Review